

Towards AI's Agent Architecture Cheat Sheet - Decision-Ready

Goal: decide **Workflow vs Single Agent vs Multi-Agent** from real constraints, then implement reliably.

Pro Tip: Check the full workflow vs. agents Webinar for free here: <https://www.youtube.com/watch?v=iOpLKlY0vXs> (and subscribe to the channel to learn more! 😊)

1) The Architecture Spectrum (Stay as Simple as Possible)

Workflow -> Single Agent + Tools -> Multi-Agent

Each step right increases cost (~4× to ~15× more tokens), latency, and debugging complexity. Stay as far left as you can while solving the problem.

The Autonomy Test

- You control steps + order -> Workflow
- Model decides what to do next (tool choice + next action) -> Agent

Tools ≠ Agents (Common Architecture Mistake)

- Tools are capabilities (API calls, DB queries, validators, browsers).
- Single LLM calls and workflows can use user defined tools following specific ("hard-coded") conditions.
- An agent is the decision-maker that chooses tools and decides the next step.
- One model calling 10 APIs is one agent with 10 tools, not multi-agent.

WHEN TO USE WHAT

2) Choose a Workflow When...

- Steps are known and stable (same order most runs).
- Predictability matters: easy unit tests per step, clear traces, deterministic gates.
- Cost/latency matters: fewer thinking loops, fewer tool decisions.

Default recommendation: If you can define the steps upfront, build a workflow first. Agents add overhead without adding capability.

Example: A support ticket pipeline where every request goes through classify → route → draft → policy check → send, in the same order every time.

3) Choose a Single Agent + Tools When...

- Tightly coupled tasks: steps are interdependent and mostly sequential (global context matters end-to-end).
- Path changes based on findings: retries, fallbacks, clarification, partial data.
- Tool load:
 - ≤10–20 *preloaded* tools → reliable selection

- >20 tools → selection degrades **unless tools are dynamically discovered** (e.g. file-based catalogs (e.g. Anthropic “Skills”, Cursor’s dynamic context discovery with MCP), tool RAG, late MCP connection)
- Low overhead desired: no inter-agent messaging/orchestration; fewer moving parts.

Example: An agent that writes code, runs it, inspects errors, fixes the code, and retries until tests pass.

4) Go Multi-Agent Only When Forced By Constraints

- True parallelism: independent subtasks must run simultaneously for throughput/speed.
- Context/tool overload: >200K tokens or >20 preloaded tools → first try dynamic tool discovery; split by domain only if reasoning must be isolated.
- Distinct competencies: phases need fundamentally different reasoning modes (e.g., exploratory research vs deterministic writing).
- Modularity/external integration: integrate third-party agent systems or reusable components.
- Hard separation: security boundaries, compliance isolation, sensitive data handling.

Example: One agent independently researches a topic in depth, while another agent focuses solely on writing a structured technical article from the research agent's output.

Multi-Agent Pattern That Usually Works

Prefer orchestrator -> worker or sequential handoff with explicit artifacts/contracts.

Avoid everyone talking to everyone (information silos + coordination failures).

When Multi-Agent Does NOT Fit

- Tasks requiring all agents to share the same context
- Many dependencies between agents requiring real-time coordination
- Most coding tasks (fewer parallelizable subtasks than research)

5) Core Engineering Rules (Make It Reliable)

- Thin agent, heavy tools: agent plans/decides; tools execute mechanics + deterministic constraints.
- Validation loops: generate -> validate -> fix with actionable feedback (hard checks first, then “reflection (similar to LLM-as-judge used in evals)).
- Observability is mandatory: trace prompts, tools, token costs, outputs; auto-evals.
- Having a human-in-the-loop (HITL) is a design choice: place human checkpoints on key artifacts (plans, research notes, drafts, before irreversible actions).

A Practical Decision Process (Use This In Kickoffs)

1. Clarify scope: deliverable (prototype vs production vs handoff), demo cadence, documentation, who maintains it.
2. Map task shape: sequential vs branching (parallel executions); exploratory (flexible) vs deterministic (fixed); single domain vs multiple domains (e.g. requiring various levels of reasoning).
3. Pick the minimum architecture: workflow first, then single agent, multi-agent only for true constraints.
4. Design tools: group by domain; enforce deterministic rules in code; return structured outputs + actionable errors.
5. Choose orchestration: a simple loop if the task is stateless; a graph or framework only if you need to persist state, branch execution paths, or pause and resume long-running work (e.g., long workflows, human-in-the-loop pauses, or resumable failures).
6. Pick models by step difficulty: strong models for planning/judgment; cheaper models for narrow execution/cleanup.
7. Document decisions: record what you chose and why (prevents rework when stakeholders change).

Shortcut: If you are uncertain, start with a workflow or a single agent with a small toolset; for exploratory phases, AI frameworks are great—then instrument and only move right once traces show a real production constraint.

The Questions That Decide What to Build (With Concrete Outcomes)

Architecture Selection (answers -> Workflow vs Single Agent vs Multi-Agent)

Q	Question	Answer patterns / signals	Build this
1	Task shape: sequential vs branching?	Sequential /linear steps -> stable order. Branching /looping -> conditional paths, retries, uncertainty.	Sequential or simple branching, looping and conditional paths -> Workflow (pipeline or graph). Advanced branching and conditions -> Agent.
2	Reasoning: exploratory or deterministic?	Deterministic : rules/constraints; known checks; strict formats. Exploratory : search/pivot, unknown next step.	Deterministic -> Workflow + tools. Exploratory -> Agent + tools (single or multi).
3	How many tool types? (tool-load)	≤5-10 tools -> easy selection. ~10-20 -> manageable if well scoped. >20 heterogeneous tools -> selection degrades.	≤10-20 -> Single agent + tools. >20 -> Split by domain (multi-agent or separate MCP/tool servers).
4	Can it decompose cleanly into distinct competencies?	Yes : phases with different reasoning modes (e.g., research vs writing). No : tightly coupled/global context.	Yes -> Multi-agent with explicit handoffs. No -> Single agent or workflow.

System Design (once architecture is chosen)

Q	Question	If you answer...	Do this
5	Need internal/proprietary data? (RAG decision)	Yes: external data needed at generation time. No: APIs/tools + model knowledge suffice.	Yes -> Build retrieval or MCP tools over internal sources. No -> Keep it simple.
6	Need persistent state ? (memory decision)	Yes: session memory, histories, intermediate artifacts must survive across steps or runs. No: all state fits in the prompt or request lifecycle.	Yes -> Use external state (DB, vector store, object storage). No -> Stateless prompts or in-memory context.
7	Need durable execution ? (workflow/framework decision)	Yes: long-running jobs, pause/resume, retries, branching recovery, human-in-the-loop. No: short-lived, synchronous execution.	Yes -> LangGraph (or durable workflow engine + persisted state). No -> Simple loop or script.
8	Do outputs require quality gates? (validation loops)	Hard constraints (length/syntax/fields) or high-stakes correctness.	Add generate-validate-fix loops; hard checks first; LLM judge second; retry limits.
9	How much HITL is needed? (approval steps) <i>Note: HITL is more often linked to UI/UX process design, not AI specifically!</i>	High HITL: humans must approve plans/research/drafts before proceeding.	High -> insert review checkpoints + editing UI. Low -> automate more; fewer gates.
10	Latency tolerance?	Tight latency: real-time UX. Loose latency: offline/batch acceptable.	Tight -> fewer hops, smaller models, more workflows, batching, caching. Loose -> deeper loops, more evaluation, multi-agent if needed.

11	What eval data exists?	Have labeled examples vs none.	Have evals -> build automated eval harness + compare prompts/models. No -> human ratings (optionally bootstrap with synthetic data if you have seed labeled examples) before over-engineering.
12	How will you do observability?	Need traces of prompts, tools, costs, outputs; debugging via timelines.	Use an observability tool (e.g., Opik) or structured logs per run; no observability = flying blind.
13	Budget per task?	Low budget vs higher budget for quality.	Low -> cheaper models, fewer tool calls, caching, workflows. Higher -> stronger reasoning models + more validation/reflection.

This decision framework (and the engineering patterns behind it) is exactly what we teach in the Towards AI courses.

Learn more in our free Agents webinar: <https://www.youtube.com/watch?v=iOpLKjY0vXs>

Learn more in our full course:
academy.towardsai.net/courses/beginner-to-advanced-llm-dev?ref=1f9b29